

01 What it is — and the problem it solves

The one-paragraph version, then the gap it closes.

The solution is a single conversational agent that turns plain-English questions into **governed, read-only queries** across the unified-communications estate. The consolidation point is an **Amazon Bedrock AgentCore Gateway acting as the MCP server**: each platform — Teams, Webex, and (in POC) ThousandEyes — is exposed to the model as a set of read-only tools, and the Gateway injects every upstream credential **server-side**, so the model never sees a Teams application secret or a Webex token. The agent runs on **AgentCore Runtime**; the model is **Amazon Bedrock (Nova)**; the front door is a browser **web console** (static SPA on S3 + CloudFront, Cognito login). One pane, plain English, no dashboard-hopping.

The gap it closes. Today a stakeholder asking "is our comms platform healthy right now, and where's the fault?" has to open the **Teams admin centre**, **Webex Control Hub**, and — separately — **ThousandEyes**: three consoles, three data models, three logins, and no correlation between them. Answering a cross-platform question ("did the Webex room that dropped also show Teams jitter, and was the network path clean?") is manual, slow, and needs a specialist in each tool. This solution consolidates that ingest behind one MCP surface and lets *anyone* ask in natural language — closing a real observability gap by **consolidating tool-target ingest via MCP** and putting an AI query layer in front of it.

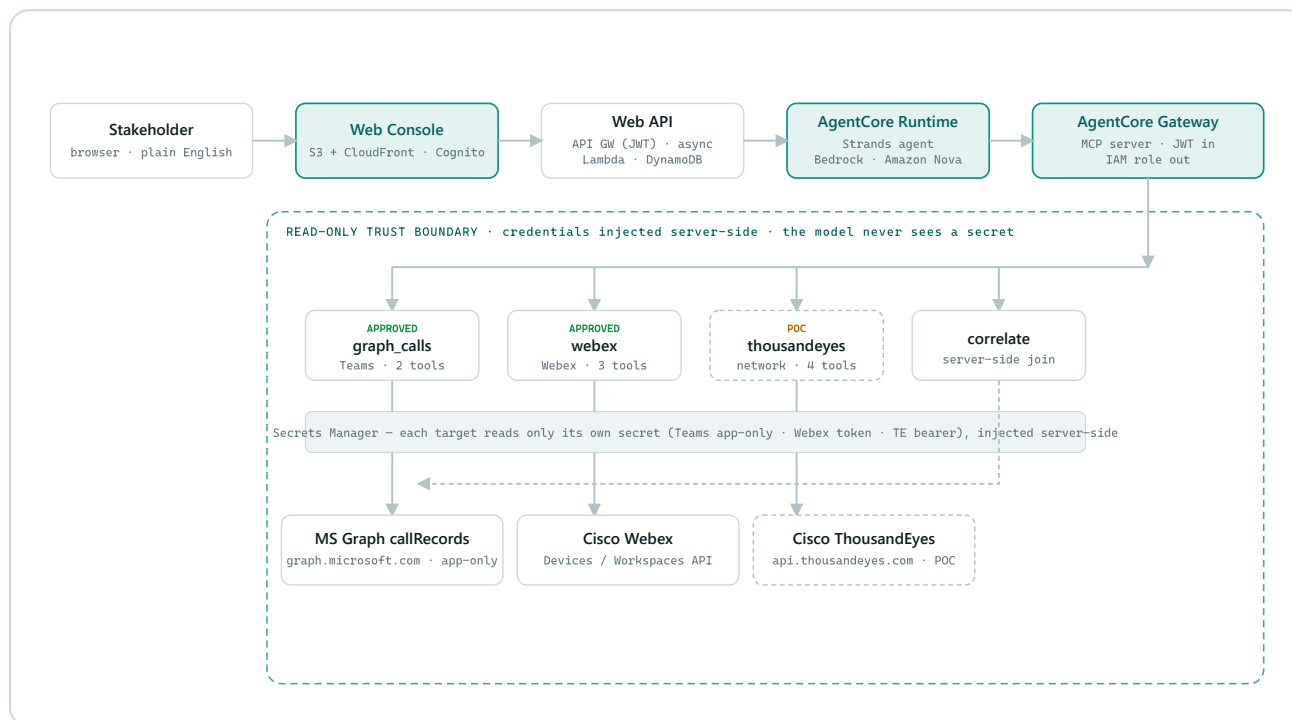
● **approved** live & approved for ANZ

● **POC** proof-of-concept · not yet approved

● **planned** specced, not built

02 The architecture — end to end

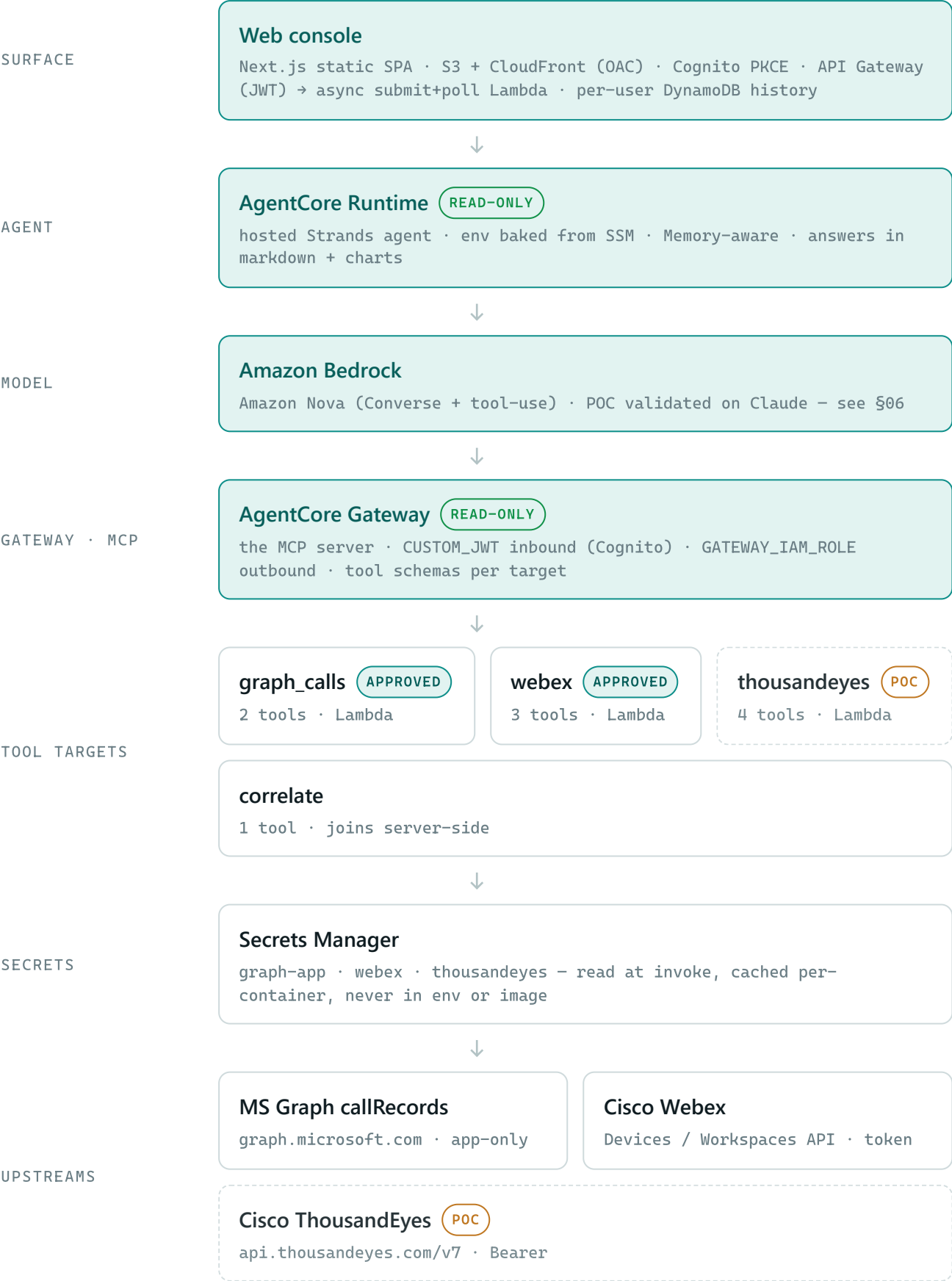
First as a flow diagram with the trust boundary made explicit, then as a top-down component map.



The load-bearing idea is the **Gateway**: it is the MCP server. Each Lambda is registered as a Gateway target with an inline tool schema, so the model discovers `graph_call_quality`, `webex_device_status` and the rest as first-class MCP tools. Inbound calls carry a JWT (Cognito today, Entra-fronted later); outbound Lambda invocation uses the Gateway's own IAM role. That credential-injection boundary is the whole security story — **upstream secrets never cross into the model context**.

Hosting — S3 origin, CloudFront delivery. The console is a static export in a **private S3 bucket**; CloudFront (Origin Access Control) is the only reader, and it's where TLS (ACM), AWS WAF, security headers and the custom domain attach. S3 static-website hosting is HTTP-only and forces the bucket public — not an option for a bank — so **CloudFront stays in front of S3**. You can host the files in S3 either way; CloudFront is the security and delivery layer, not an optional CDN.

The same stack, top-down // component map



Approval & integration status

INTEGRATION	ANZ STATUS	AUTH	WHAT IT GIVES THE AGENT
Microsoft Teams (MS Graph)	● approved	App registration · app-only (<code>CallRecords.Read.All</code>)	Teams call records + per-session call quality
Cisco Webex	● approved	Integration / service-app API key (read-only)	Device connection status + workspace environmentals
Cisco ThousandEyes	● POC · not approved	Bearer token (v7 API)	Network / voice test results + path visualisation

03 The intent — why we need this

Three reasons, in the order that matters to a sponsor.

- ◆ **It aligns with ANZ's enterprise-AI direction.** This is agentic AI on *approved rails* — Amazon Bedrock for the model, the agent hosted in-account on AgentCore, identity through Cognito/Entra, secrets in Secrets Manager, and **read-only by construction**. There is no create/update/delete path anywhere in scope. That is the shape of AI a risk and controls function can actually sign off, rather than a shadow tool reaching production systems.
- ◆ **It closes a genuine observability gap.** Platform and service health for unified comms is scattered across Teams, Webex and the network layer. Consolidating that ingest behind one MCP surface, and querying it in plain English, turns a multi-console, specialist-only task into a single question anyone can ask — at any time.
- ◆ **The pattern is repeatable — that's the real prize.** The Gateway-as-MCP plus web-console pattern is domain-agnostic. Any read-only enterprise data source — infrastructure, security signals, payments telemetry, the service desk — can be onboarded as another *tool target* and surfaced through the **same AWS-hosted web console**. Prove the model on unified comms; then reuse the Gateway and the console for the next domain without re-litigating the security architecture.

The value isn't the three integrations — it's the pattern. A governed, read-only, in-account agentic-AI front door that any approved data source can plug into, and any stakeholder can query in natural language. Unified comms is the first, defensible proof point.

04 **The tool surface — what the MCP actually calls**

Every call the agent can make is a read-only, GET-shaped tool. Grouped by target; ThousandEyes rows are POC.

TOOL	TARGET	UPSTREAM	RETURNS
graph_list_call_records	graph_calls	MS Graph	Teams call records started in a UTC window
graph_call_quality	graph_calls	MS Graph	per-session jitter / packet loss / RTT / codec
webex_list_devices	webex	Webex Devices API	registered devices + product + connection status
webex_device_status	webex	Webex Devices API	per-device connectivity, active interface, IP, software, health
webex_workspace_metrics	webex	Webex Workspaces API	per-workspace environmentals — temperature, humidity, ambient noise, people count
te_list_tests_alerts	thousandeyes ● POC	ThousandEyes v7	configured tests + active alerts
te_network_results	thousandeyes ● POC	ThousandEyes v7	loss / latency / jitter per agent
te_voice_results	thousandeyes ● POC	ThousandEyes v7	MOS / jitter / loss / latency (RTP)
te_path_visualization	thousandeyes ● POC	ThousandEyes v7	hop-by-hop path with per-hop latency / loss

TOOL	TARGET	UPSTREAM	RETURNS
<code>correlate_workspace_calls</code>	correlate	joins server-side	Webex workspace / device health joined to its Teams calls' quality (adds TE path once approved)

Every tool maps to a single upstream `GET`. There is no tool that writes, provisions, or changes state — the surface is the security argument made concrete: the worst an over-eager model can do is read something it was already entitled to read.

05 Logical view — one request, end to end

The same picture as a lifecycle, so the trust boundaries are explicit.

- ◆ **Ask.** The stakeholder types plain English into the web console. The web tier mints a JWT and opens a streamable-HTTP MCP session to the Gateway on the user's behalf.
- ◆ **Discover.** The model receives the Gateway's `tools/list` — the tool specs with JSON schemas — as its tool config.
- ◆ **Reason + call.** Bedrock (Nova) decides which tools to call. Each call is a JWT-authorized MCP request to the Gateway, which invokes the target Lambda under its own IAM role.
- ◆ **Fetch, secretly.** The Lambda reads only *its* secret from Secrets Manager, calls the upstream, and returns compact JSON. The secret never leaves the function.
- ◆ **Answer.** Tool output re-enters the model as untrusted context; the loop repeats until the model stops calling tools, then it returns prose — with the standing instruction that correlation is *observational, not causal*.

Two independent auth hops, by design. Caller → Gateway is one identity protecting the MCP endpoint. Gateway → Lambda → upstream is per-service secret injection. Collapsing these — e.g. handing a Webex token or the Teams app secret to the browser — is the exact anti-pattern this architecture exists to avoid.

CREDENTIAL INJECTION · ONE PATTERN PER TARGET

```
# The model never sees any upstream credential. Each Lambda r
# own secret, calls the upstream, and returns compact JSON.
def webex_get(path, params=None):
    sec = get_secret(os.environ["WEBEX_SECRET_ARN"])          #
    status, payload = _http("GET", f"{WEBEX_BASE}{path}", {
        "Authorization": f"Bearer {sec['access_token']}",      #
        "Accept": "application/json",
    })
    return payload      # compact - it re-enters the model as un
```

06 Things to look out for & consider

Ordered by what a bank's risk and platform teams raise first. Severity: red = block, amber = fix before scale, green = hygiene.

SEVERITY	TITLE	DESCRIPTION	ACTION	OUTCOME
BLOCK	Data residency	The POC runs in <code>us-east-1</code> . Teams, Webex and network telemetry landing offshore is a sovereignty issue for an ANZ banking audience.	Deploy Runtime + Gateway + Bedrock Nova in <code>ap-southeast-2</code> (Sydney); confirm each component's availability in-region.	Comms data stays in-country; sovereignty question closed.
BLOCK	Teams grant is tenant-wide	<code>CallRecords.Read.All</code> is app-only and reads <i>every</i> Teams call in the tenant — effectively an org-wide key. <code>callRecords</code> does not support delegated auth.	Use a dedicated app registration used by nothing else; scheduled rotation; Secrets Manager only; document in the risk register with a named owner.	A scoped, owned, auditable grant with a known blast radius.
BLOCK	Approved model — validate on Nova	The POC ran on Claude; ANZ's approved Bedrock model set may be Amazon Nova only. Nova supports tool use, but agentic fidelity on this loop must be proven, not assumed.	Run the tool-use loop on Nova (Pro for reasoning, Lite for simple status) and score it against a fixed eval set before committing.	Solution runs on an approved model with measured quality.
FIX BEFORE SCALE	ThousandEyes not yet approved	TE is a POC axis. It must not carry data in a production path until it clears approval.	Keep TE behind a feature flag / non-prod boundary; exclude its Lambda, secret and IAM from the production stack until approved.	No unapproved data flow; a clean, staged approval path.

SEVERITY	TITLE	DESCRIPTION	ACTION	OUTCOME
FIX BEFORE SCALE	Single-operator inbound auth won't scale	A shared identity / one client is fine for a POC. The moment more than one human or a UI needs access, you need real per-user identity, not shared IAM.	Adopt AgentCore Identity for per-user inbound OAuth, fronted by Entra SSO, before opening access.	Real per-user identity and an audit trail.
FIX BEFORE SCALE	Tool output is untrusted context	Upstream strings (device names, call subjects, test labels) re-enter the model — a prompt-injection surface. Low risk on first-party data, non-zero at a bank.	Add Bedrock Guardrails (PII redaction + injection heuristics); keep tool payloads compact (already done).	Reduced injection and PII surface ahead of security review.
FIX BEFORE SCALE	Webex token scope	A broad Webex token could read or do more than device/workspace status. Least privilege must be explicit.	Scope the Webex integration / service app to read-only device + workspace scopes; store the token in Secrets Manager only.	Least-privilege upstream access, defensible in review.
HYGIENE	Secret cache outlives rotation	A per-container cache serves the old secret after rotation until the warm container recycles.	Add a short TTL (5–15 min) to the secret cache.	Rotation propagates without a cold start or confusing outage.
HYGIENE	No fleet observability yet	Only container stdout today — no per-session traces, tool latency, or token spend across users.	Enable AgentCore Observability (OTEL → CloudWatch GenAI) — near-zero effort.	Traces, latency, token spend and error rates, fleet-wide.

07 Optimisations worth doing

Cheap wins first. None of these change the read-only contract. Effort as signal bars:

4 green = drop-in, 3 green = a day.

OPTIMISATION	EFFORT	WHAT IT DOES	OUTCOME
Prompt-cache the static prefix	 DROP-IN	The system prompt and tool schemas are identical every turn; cache that prefix in Bedrock.	Materially lower input-token cost on a chatty tool-use loop.
AgentCore Observability	 DROP-IN	Strands emits OpenTelemetry natively; point it at CloudWatch GenAI.	Per-session traces, tool latency and token spend, with no code change.
Right-size the Nova tier	 ~A DAY	Route simple status queries to Nova Lite/Micro, complex correlation to Nova Pro.	Lower cost per query without losing quality on the hard questions.
Gateway semantic tool search	 ~A DAY	As targets grow, load only the relevant tools into the prompt instead of the full list.	Smaller prompts — lower latency and cost as the estate expands.
arm64 (Graviton) Lambdas	 ~A DAY	Run the pure-Python targets on Graviton via a Linux/CodeBuild build step.	~20% cheaper for the same workload, consistent builds.
Webex via OpenAPI Gateway target	 ~A DAY	Wrap the Webex REST API with an OpenAPI spec handed to the Gateway — no handler code.	Faster onboarding and less bespoke code to own and secure.
correlate: TTL cache + parallel fetch	 ~A DAY	Cache device/workspace lists and fan out to upstreams concurrently.	Shortens the slowest path in the system on correlation queries.

08 Toolsets to add — and complexity for delivery

Effort read as signal bars: 4 bars green = drop-in, 3 green = a day, 2 amber = a sprint, 1 red = a project.

 drop-in  ~a day  a sprint  a project

AgentCore Observability

OPS · OTEL → CLOUDWATCH

Dashboards, tracing and token metrics with no code change — Strands is already OTEL-native. The single highest-value add.

 DROP-IN → [config template](#)

Bedrock Guardrails

SAFETY · PII + INJECTION

Redact PII from tool output and dampen prompt-injection from untrusted upstream strings (\$06). A near-requirement before a banking security review.

 ~A DAY → [guardrail policy](#)

AgentCore Code Interpreter

ANALYSIS · SANDBOXED PYTHON

Let the agent *compute* over correlated data — a real correlation coefficient between Webex workspace noise and Teams jitter — instead of eyeballing it.

 ~A DAY → [tool-enable template](#)

Cisco Catalyst Center / Meraki

SOURCE · OPENAPI TARGET

Extend the network axis beyond ThousandEyes — wired/wireless health for the sites the comms estate depends on. Onboarded as an OpenAPI target, no handler code.

 ~A DAY → [spec + target](#)

ServiceNow / ITSM

SOURCE · OPENAPI TARGET

Tie comms health to the incident and change record — "is this drop already a known incident, and did a change precede it?" Read-only queries into ITSM.

 ~A DAY → [spec + target](#)

ThousandEyes — MCP federation

SOURCE · MCP TARGET

Once TE is approved, federate its hosted MCP instead of the hand-wrapped Lambda. *Trade-off:* it exposes write verbs, so keep the read-only Lambda or filter capabilities.

 ~A DAY → [MCP-target template](#)

AgentCore Identity + Entra SSO

AUTH · PER-USER OAUTH

Per-user inbound tokens (replaces single-operator access) and managed outbound OAuth/token refresh. The unlock for multi-user, audited access.



A SPRINT

→ [identity wiring](#)

AgentCore Evaluations

QUALITY · REGRESSION TESTS

A fixed question set scored against expected answers, run on every change — so a model bump (Claude → Nova) or prompt edit can't silently regress a known-good answer.



A SPRINT

→ [eval-suite template](#)

Multi-agent (Strands GraphBuilder / A2A)

SCALE · AGENT-TO-AGENT

Split network / workspace / call reasoning into specialist agents behind a supervisor. Powerful, but only earns its keep once the single agent is genuinely overloaded. Start monolithic.



A PROJECT

→ [later](#)

Each toolset above is an `agent/` or `infra/` drop-in matched to the existing repo conventions (SAM for Lambda / Cognito, boto3 scripts for AgentCore resources). The three current integrations – Teams, Webex, and the ThousandEyes POC – already follow this exact pattern, which is why a fourth source is a day's work, not a rebuild.

AWS Bedrock AgentCore — Unified Communications Observability. Prepared for ANZ. Architecture and status reflect the deployed proof-of-concept; ThousandEyes is a non-approved POC axis shown for completeness. Model shown as Amazon Nova (target); the POC was validated on Claude — model selection is an open decision (\$06).

Amazon Bedrock AgentCore – overview

AgentCore Gateway concepts

Gateway OpenAPI targets

AgentCore Observability

AgentCore pricing

Cisco Webex – Devices API

Cisco Webex – Workspaces API

MS Graph – callRecords API

Cisco ThousandEyes – v7 API

Deploy Strands to AgentCore Runtime

Styling and signal-bar language inherited from the neat-graph-bedrock reference architecture. No account or resource identifiers are included in this document.